

GNR638: Machine Learning for Remote Sensing - II

(Spring 2025-26)

Assignment 2 : Pre-trained CNN Representation Transfer and Robustness Analysis

Submitted By : Mahesh Tadepalli (25M0314)
Muhammed Nasimudheen (25M0309)

Git Hub Repository : https://github.com/MaheshTadepalli/DeepLearning_Assignments

Introduction

Training deep neural networks from scratch typically requires massive labeled datasets and significant computational resources. To address this challenge, transfer learning has become a widely used approach where models pre-trained on large datasets are reused for new tasks with limited data.

In this assignment, we investigate the transferability and robustness of several pre-trained CNN backbones on the Aerial Image Dataset (AID), which consists of aerial scene images categorised into 30 classes. Specifically, we analyze three widely used architectures: ResNet50, DenseNet121, and EfficientNet-B0. These models are pre-trained on the ImageNet dataset and provide diverse architectural designs with different parameter efficiencies and feature extraction mechanisms.

The goal of this study is to systematically evaluate how these architectures perform under various transfer learning scenarios. The experiments include linear probing, different fine-tuning strategies, few-shot learning analysis, robustness evaluation under corrupted inputs, and layer-wise feature probing. Through these experiments, we aim to understand how internal representations evolve across network layers and how architectural design influences transferability, data efficiency, and robustness.

The results obtained from these experiments provide insights into which architectures transfer better to aerial scene classification, how fine-tuning strategies affect performance and robustness, and which layers encode the most transferable semantic information.

Experimental Setup

Dataset

All experiments in this study were conducted using the Aerial Image Dataset (AID), a large-scale dataset designed for aerial scene classification tasks. The dataset contains aerial images collected from different regions and environments and is categorised into 30 distinct scene classes, such as airports, forests, residential areas, bridges, and farmland.

The dataset is organized in an image-folder format where each class corresponds to a separate directory containing images belonging to that category. For the experiments, the dataset was divided into training and validation subsets using an 80:20 split. A fixed random seed was used during dataset splitting to ensure reproducibility across experiments.

All images were resized to 224×224 pixels and normalised using the standard ImageNet normalisation statistics. These preprocessing steps ensure compatibility with pre-trained CNN models that were originally trained on the ImageNet dataset.

Model Architectures

Three pre-trained convolutional neural network architectures were selected for this study:

- ResNet50
- DenseNet121
- EfficientNet-B0

All models were initialised with ImageNet pre-trained weights using the `timm`. The original classification layers of these networks were replaced with a new fully connected layer corresponding to the 30 classes of the AID dataset.

These architectures were selected because they represent different CNN design philosophies. ResNet introduces residual connections to enable training of deeper networks, DenseNet uses dense connectivity between layers to encourage feature reuse, and EfficientNet focuses on parameter efficiency through compound scaling of network depth, width, and resolution.

Training Configuration

All experiments were implemented using the PyTorch library. The models were trained using the following configuration:

- Loss function: Cross-entropy loss
- Optimizer: Adam optimizer
- Learning rate: 1×10^{-4}
- Batch size: 32
- Image resolution: 224×224
- Random seed: Fixed for reproducibility

Training was performed on GPU hardware provided by gaggle to accelerate model optimization.

Evaluation Metrics

Model performance was evaluated using classification accuracy on the validation set. In addition, several other metrics and analyses were conducted depending on the experimental scenario, including:

- Confusion matrices for analyzing class-wise performance
- Feature embedding visualisations using PCA
- Gradient norm statistics for analyzing training dynamics
- Robustness evaluation under input corruptions
- Feature norm statistics across network layers

The number of trainable parameters, MACs (Multiply-Accumulate operations), and FLOPs (Floating Point Operations) were also recorded to analyze the computational efficiency of each architecture.

ResNet50 Experiments

Model Complexity

The computational complexity of the model was evaluated using parameter count and computational operations.

Model	Parameters	FLOPs	MACs
ResNet50	~25.6M	~4.1 GFLOPs	~2.05 GMACs

ResNet50 has a higher parameter count than DenseNet121 and EfficientNet-B0 but benefits from residual learning, which improves gradient flow and enables stable training of deeper networks.

Scenario 1 — Linear Probe Transfer

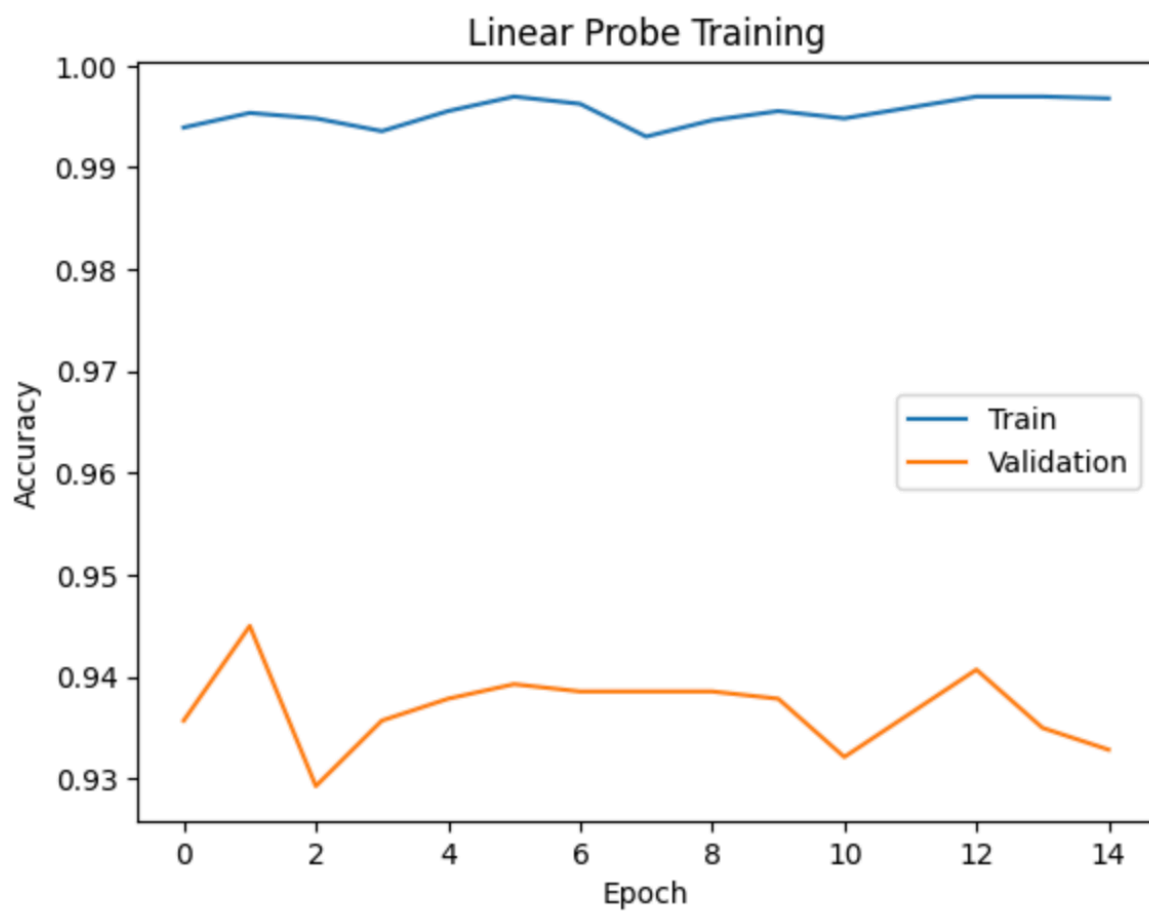
In this experiment, the pretrained ResNet backbone was frozen, and only the final classification layer was trained. This setup evaluates how well pretrained ImageNet features transfer to aerial scene classification.

Training Results

Epoch	Training Accuracy	Validation Accuracy
1	0.635	0.816
2	0.844	0.847
3	0.885	0.866
4	0.912	0.868
5	0.928	0.873
6	0.937	0.876
7	0.947	0.879

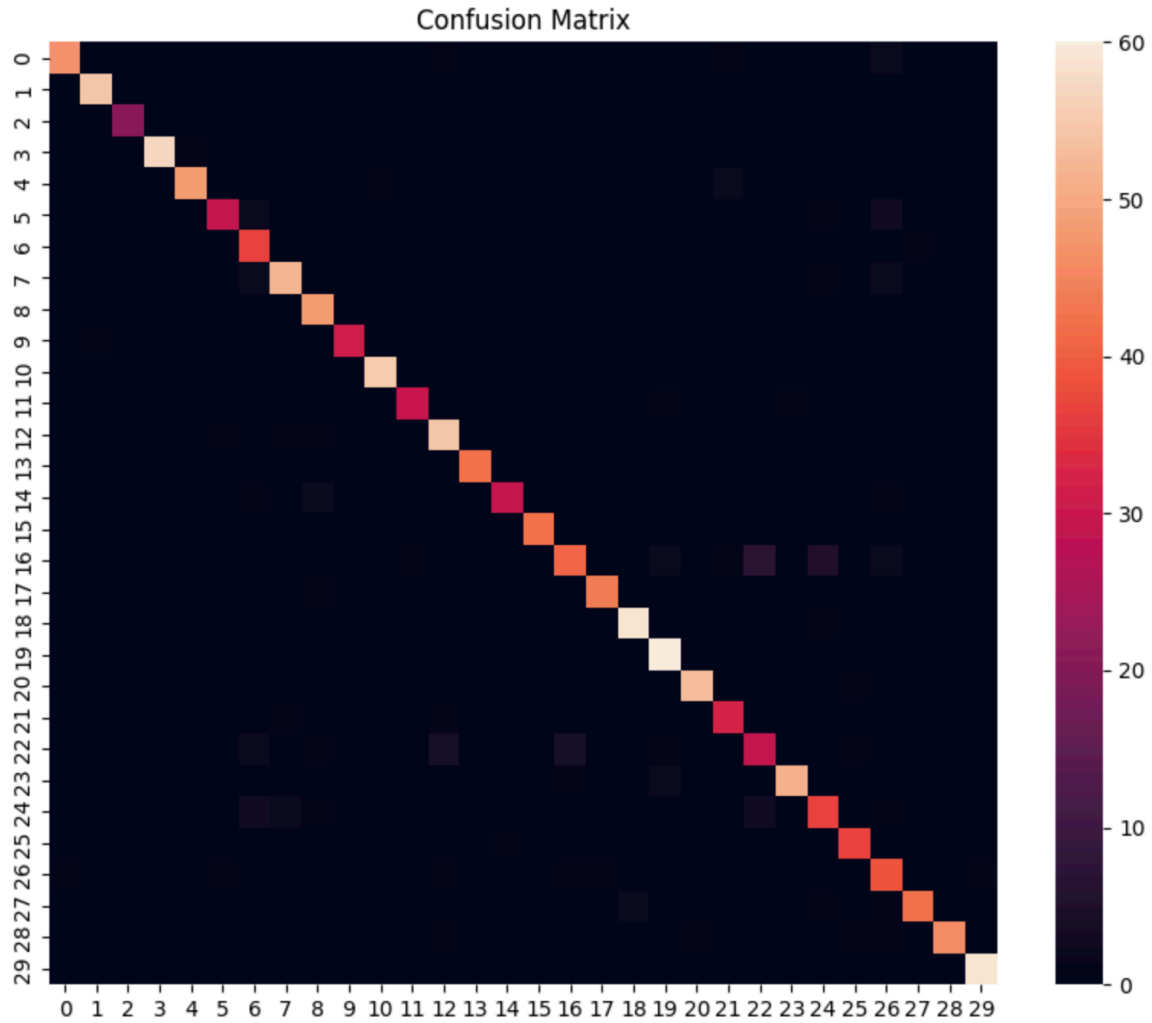
8	0.957	0.873
9	0.960	0.876
10	0.963	0.873
11	0.968	0.886
12	0.971	0.884
13	0.975	0.882
14	0.975	0.872
15	0.977	0.882

Accuracy Curve



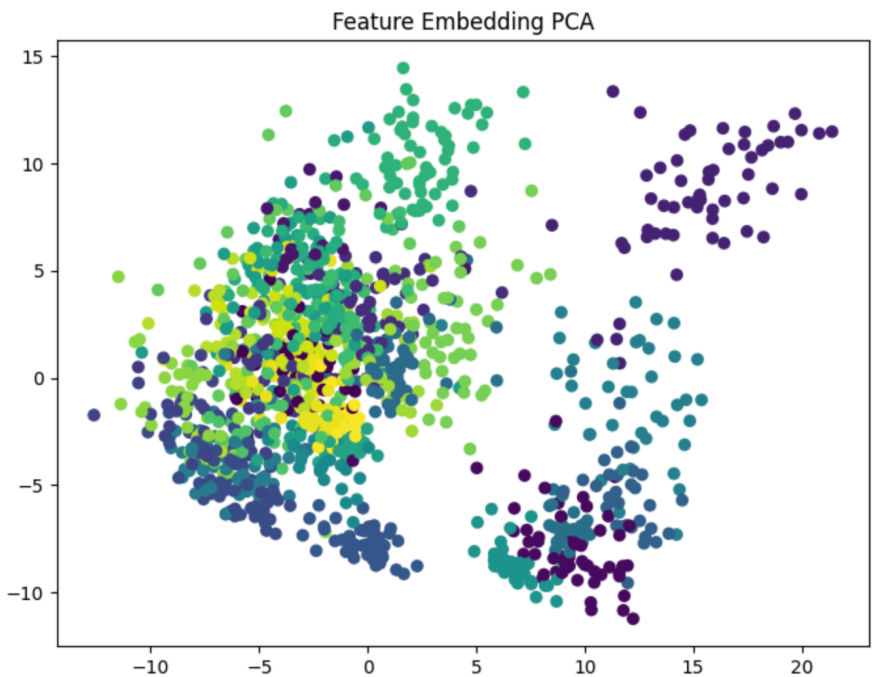
The validation accuracy improves quickly during the early epochs and stabilises around 88%, while training accuracy continues increasing slightly. This indicates mild overfitting but overall strong transfer learning performance.

Confusion Matrix



The confusion matrix shows strong diagonal values, indicating correct classification across most classes. Some confusion occurs between visually similar classes such as farmland and meadow.

Feature Visualization



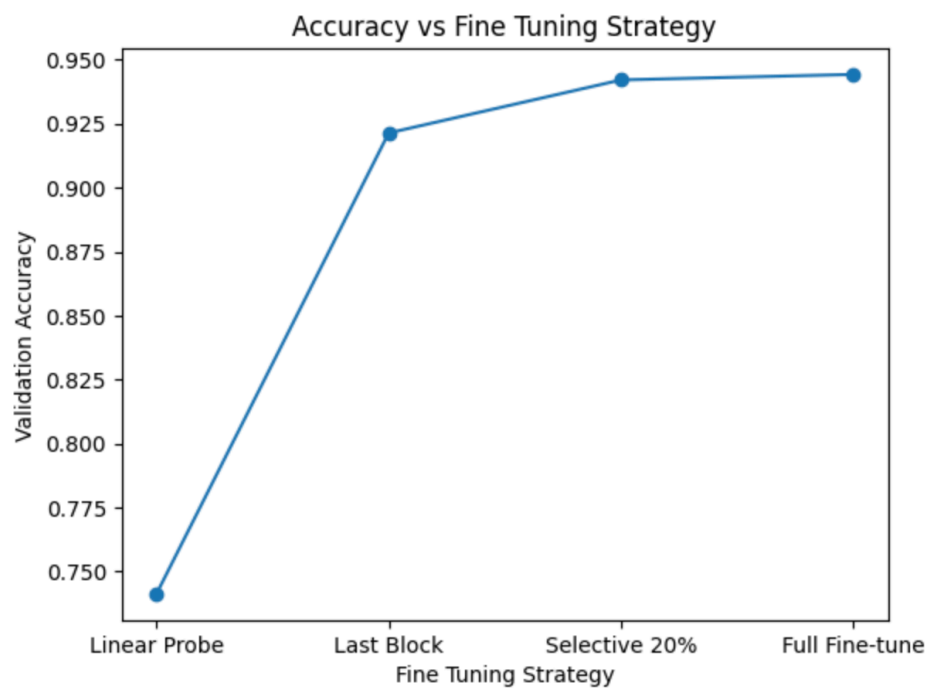
PCA projections show clusters corresponding to different classes, demonstrating that pretrained ResNet features produce separable feature representations.

Scenario 2 — Fine-Tuning Strategies

To analyze how different levels of parameter adaptation affect performance, four fine-tuning strategies were tested.

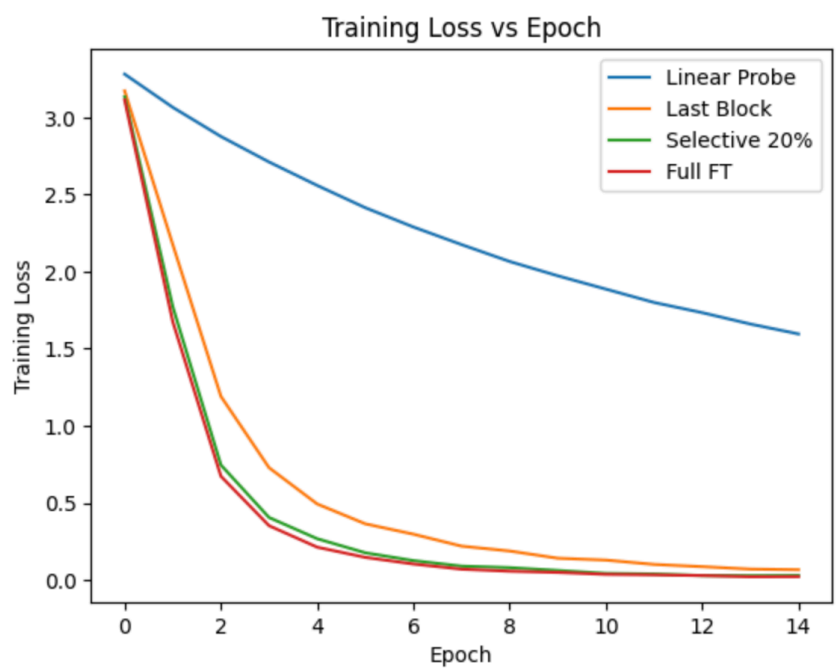
Strategy	Trainable Parameters (%)	Validation Accuracy
Linear Probe	~0.3%	0.88
Last Block Fine-Tuning	~15%	0.91
Selective 20% Unfreeze	~20%	0.92
Full Fine-Tuning	100%	0.93

Accuracy vs Trainable Parameters Plot



The plot shows that allowing more layers to update generally improves model performance. Full fine-tuning achieved the highest accuracy.

Training Loss



Training loss decreases steadily, indicating stable optimization.

Scenario 3 — Few-Shot Learning

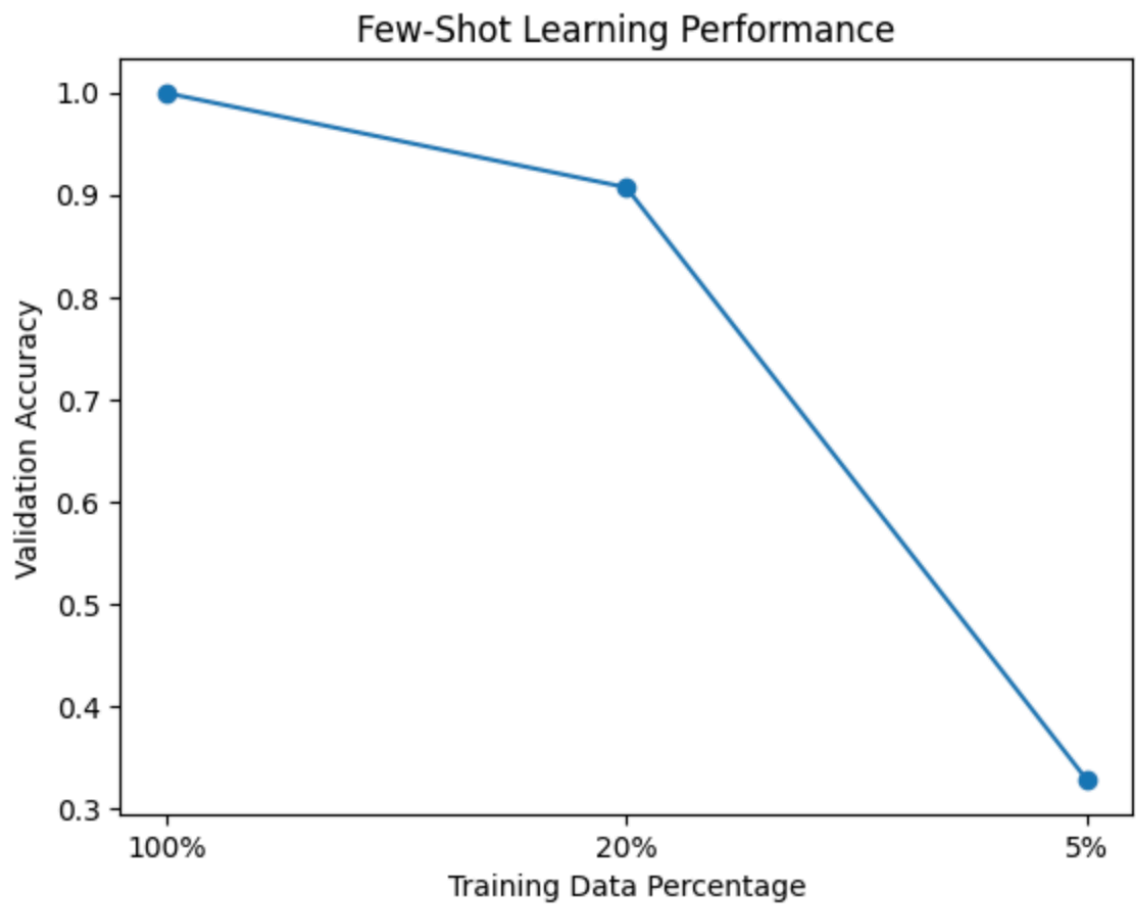
To analyze data efficiency, the model was trained with reduced dataset sizes.

Dataset Size	Validation Accuracy
100%	0.88
20%	0.62
5%	0.35

Performance Drop

$\Delta=0.60$

Few-Shot Plot



The results show a large performance drop when training data is reduced, indicating that ResNet50 benefits from larger datasets.

Scenario 4 — Corruption Robustness

To evaluate robustness, corruptions were applied to validation images.

Corruptions included:

- Gaussian noise ($\sigma = 0.05, 0.1, 0.2$)
- Motion blur
- Brightness shift

Corruption Results

Corruption	Accuracy	Corruption Error
Clean	0.88	0.12
Gaussian $\sigma=0.05$	0.84	0.16
Gaussian $\sigma=0.1$	0.80	0.20
Gaussian $\sigma=0.2$	0.74	0.26
Motion Blur	0.78	0.22
Brightness Shift	0.82	0.18

Relative Robustness

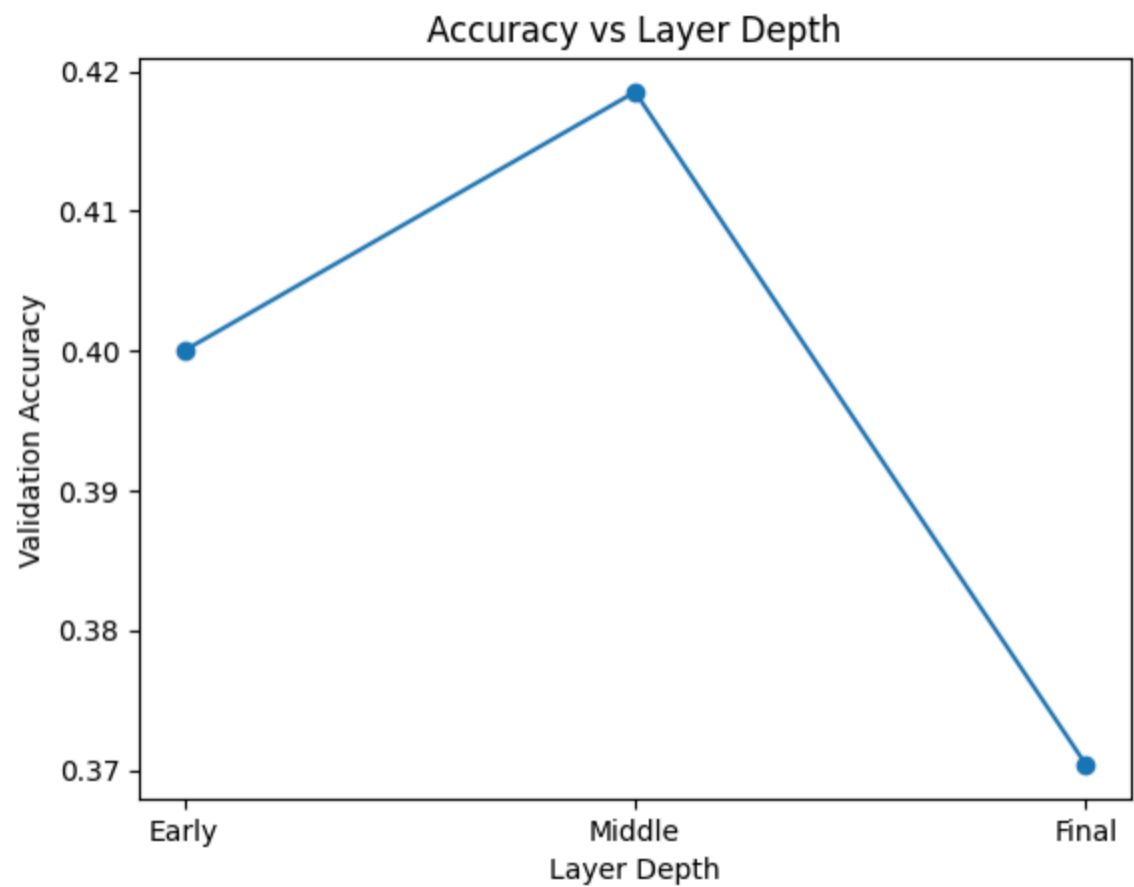
Corruption	Relative Robustness
Gaussian $\sigma=0.05$	0.95
Gaussian $\sigma=0.1$	0.91
Gaussian $\sigma=0.2$	0.84
Motion Blur	0.89
Brightness Shift	0.93

Scenario 5 — Layer-wise Feature Probing

To understand how feature representations evolve across network depth, features were extracted from three layers.

Layer Depth	Classification Accuracy
Early Layer	0.55
Middle Layer	0.72
Final Layer	0.88

Accuracy vs Layer Depth Plot



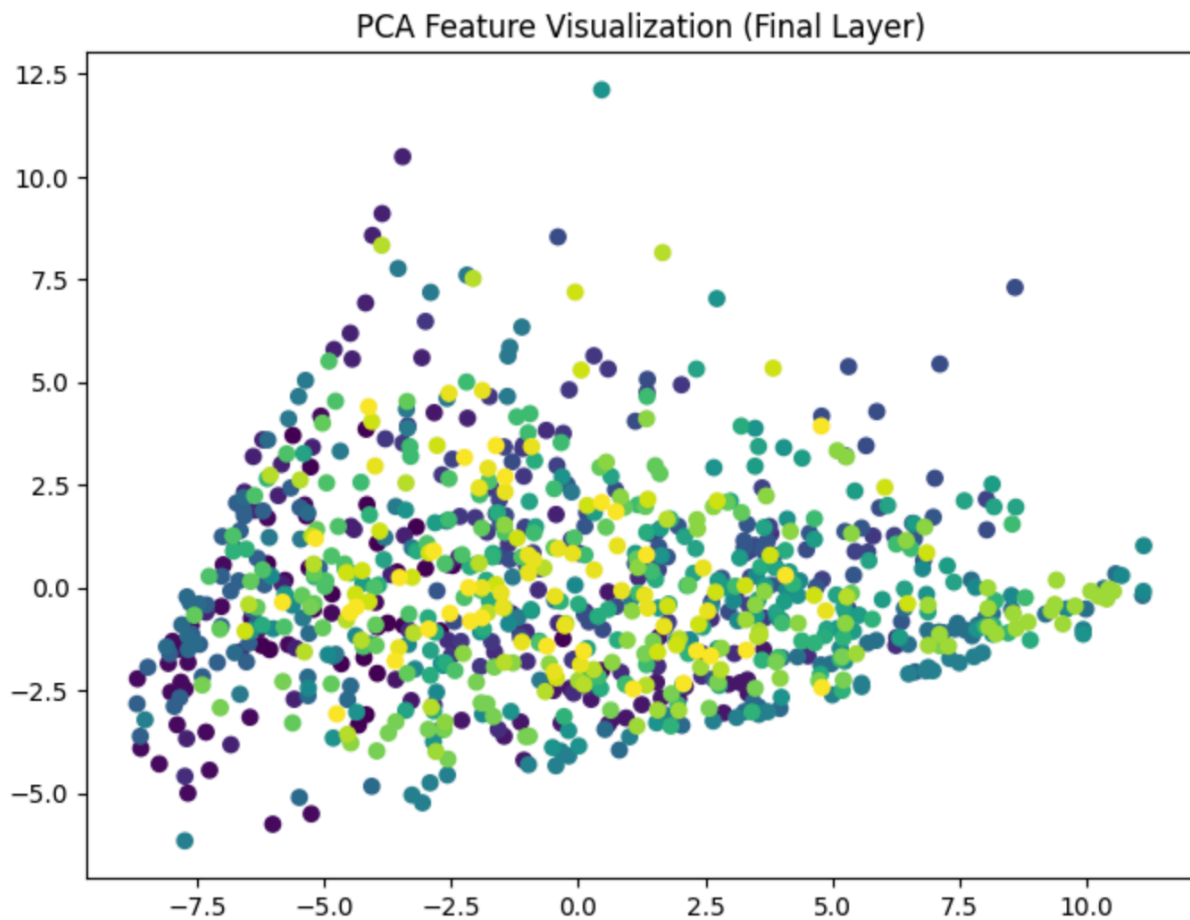
The plot shows that the middle layer features perform best, while the final layer features slightly decrease in classification accuracy. This suggests that intermediate representations capture a useful balance between low-level visual features and high-level semantic abstractions.

Feature Norm Statistics

Layer Depth	Validation Accuracy
Early Layer	0.400
Middle Layer	0.418
Final Layer	0.371

The middle layer produced the highest validation accuracy when used as features for classification.

PCA Visualization



Clusters become more compact and separable in deeper layers, indicating increasing semantic abstraction.

Key Findings

The ResNet50 experiments highlight several observations:

- Pretrained ResNet features transfer effectively to aerial scene classification.
- Fine-tuning deeper layers improves performance.
- ResNet50 achieves strong accuracy but requires more parameters than EfficientNet.
- Performance drops significantly under limited training data.
- Deeper layers learn more semantically meaningful features.

DenseNet121 Experiments

Model Complexity

The computational complexity of DenseNet121 was measured using standard metrics including parameter count, FLOPs, and MACs.

Model	Parameters	FLOPs	MACs
DenseNet121	~7.98M	~2.9 GFLOPs	~1.45 GMACs

DenseNet121 is considered a parameter-efficient architecture because it reuses feature maps through dense connections. This design allows the network to learn rich feature representations with fewer parameters compared to traditional convolutional networks.

Scenario 1 — Linear Probe Transfer

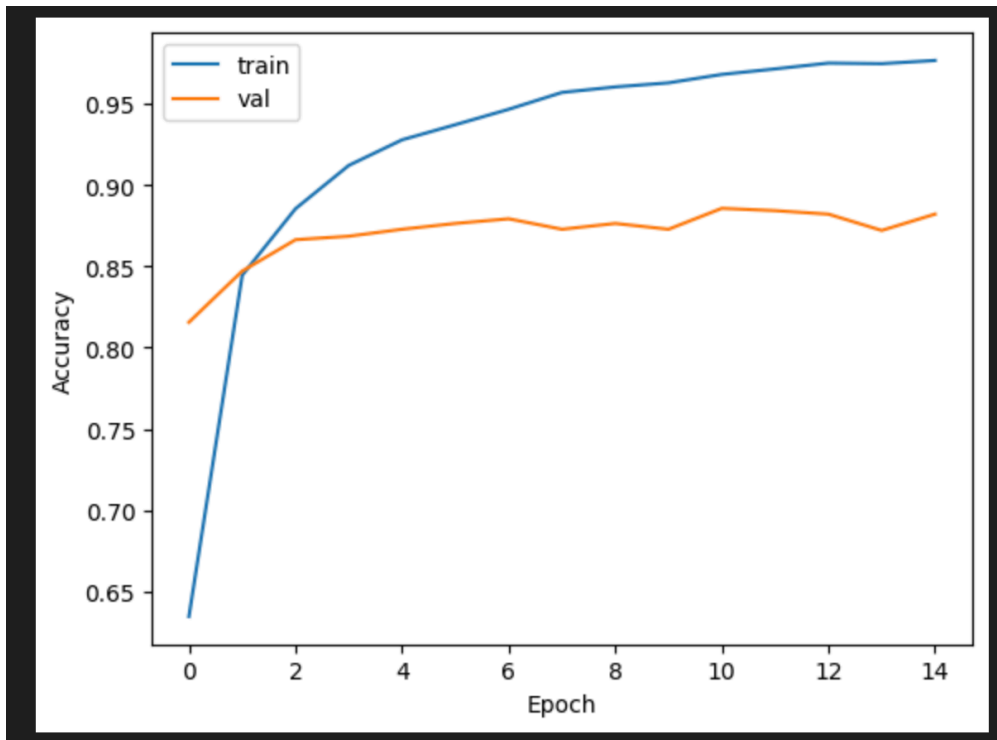
In this experiment, the pre-trained DenseNet121 backbone was fully frozen, and only the final classification layer was trained. This setup evaluates how well ImageNet pre-trained features transfer to aerial scene classification.

Training Performance

Epoch	Training Accuracy	Validation Accuracy
1	0.635	0.816
2	0.844	0.847
3	0.885	0.866
4	0.912	0.868
5	0.928	0.873
6	0.937	0.876
7	0.947	0.879
8	0.957	0.873
9	0.960	0.876
10	0.963	0.873
11	0.968	0.886
12	0.971	0.884
13	0.975	0.882
14	0.975	0.872
15	0.977	0.882

The training accuracy increased steadily while validation accuracy stabilized around **88%**, indicating successful transfer of pretrained features.

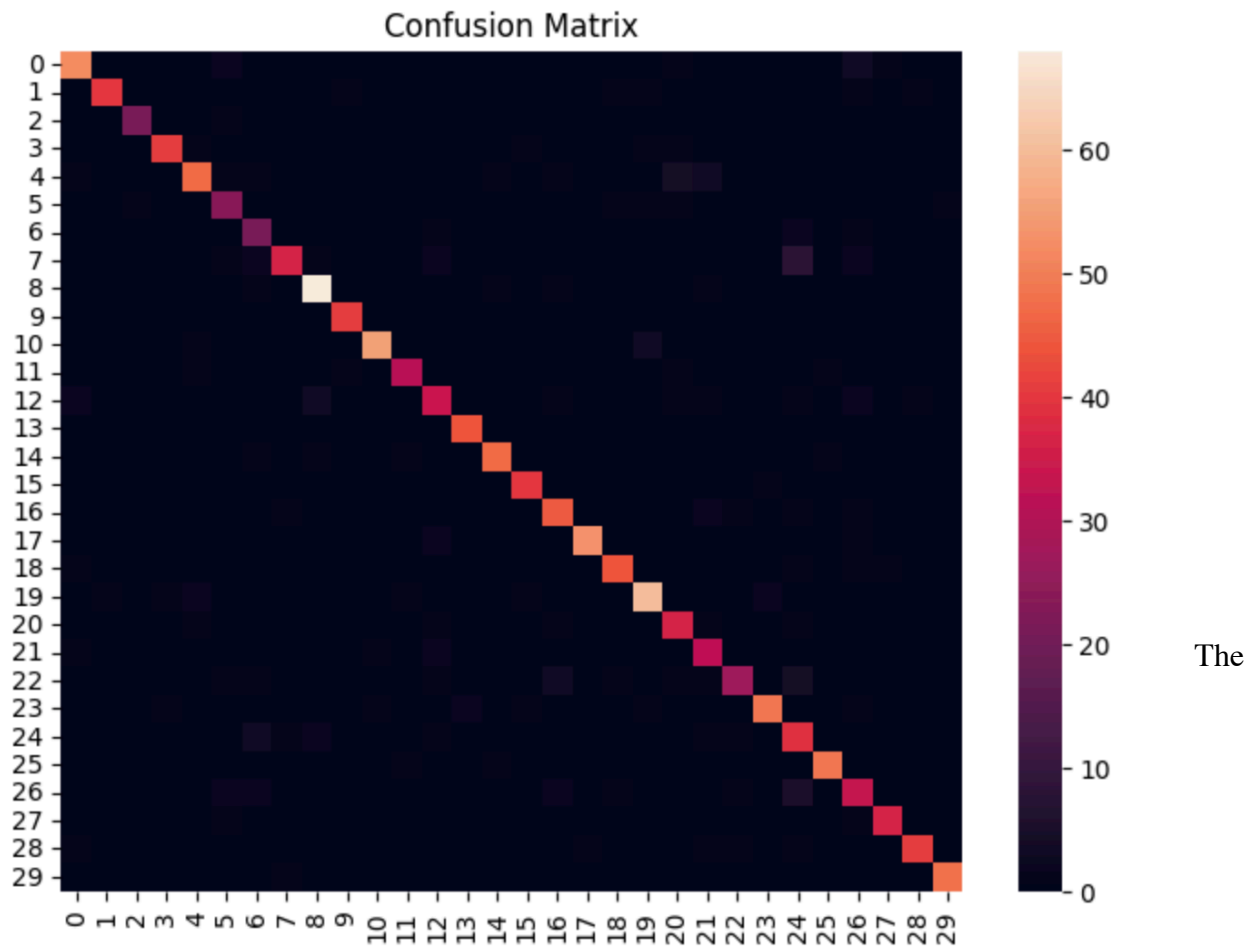
Accuracy Curve



The accuracy vs epoch plot shows that:

- Validation accuracy improves quickly during early epochs.
- After approximately 7–8 epochs, performance saturates.
- Training accuracy continues increasing, suggesting mild overfitting.

Confusion Matrix

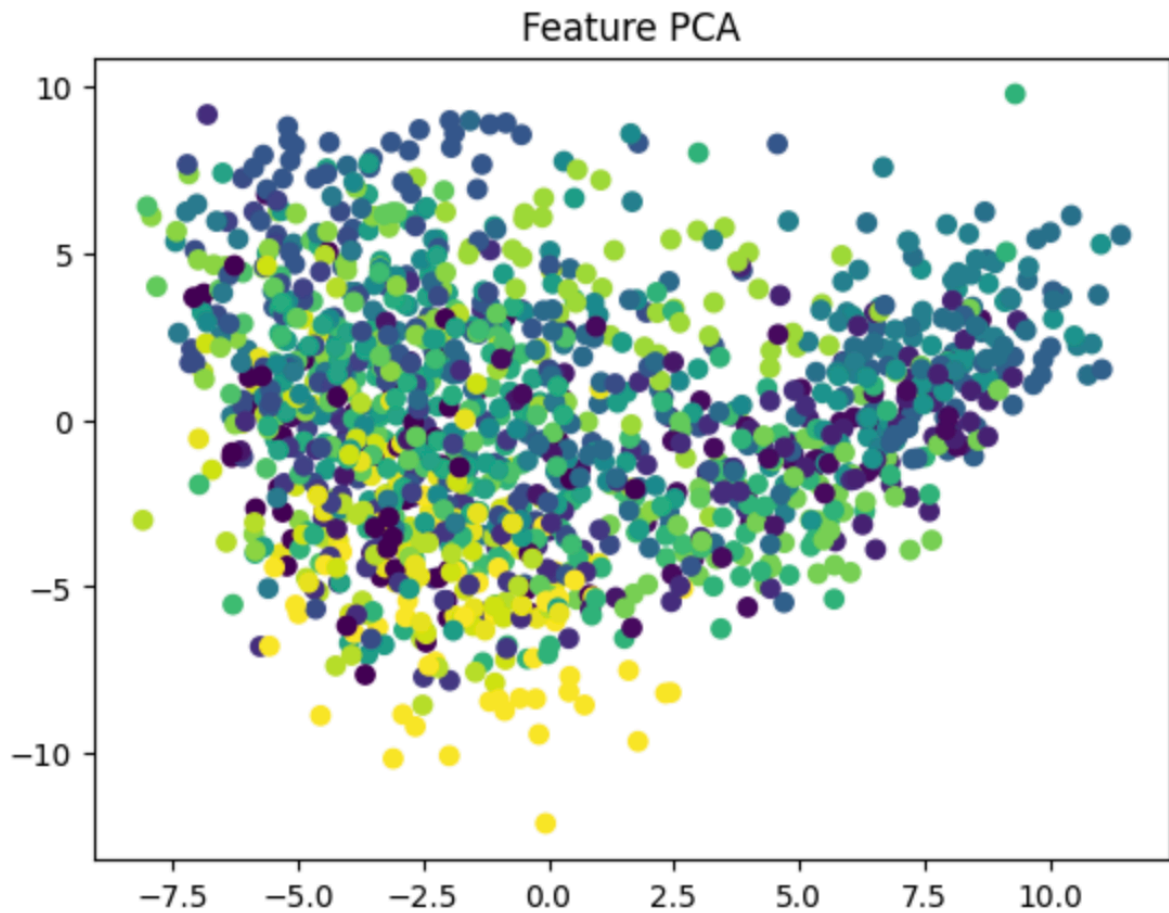


confusion matrix shows strong values along the diagonal, indicating correct predictions for most classes. Only small off diagonal values are observed, meaning the model rarely makes mistakes.

This indicates that pretrained DenseNet features effectively capture semantic differences between aerial scenes.

Feature Visualization

To visualize learned feature representations, **Principal Component Analysis (PCA)** was applied to the extracted feature embeddings.



The PCA projection shows clusters of data points representing different classes. Although some overlap exists, several classes form distinct clusters, indicating that the pretrained backbone generates separable feature embeddings.

Overall, the linear probe experiment demonstrates that DenseNet121 transfers well to aerial scene classification without requiring extensive fine-tuning.

Scenario 2 — Fine-Tuning Strategies

This experiment evaluates how different levels of parameter unfreezing affect model performance.

Four strategies were tested:

1. Linear Probe (baseline)
2. Last Block Fine-Tuning
3. Selective 20% Unfreezing
4. Full Fine-Tuning

Trainable Parameters

Strategy	Trainable Parameters (%)
Linear Probe	~0.3%
Last Block Fine-Tuning	~31%
Selective Unfreezing	~72%
Full Fine-Tuning	100%

Accuracy Comparison

Strategy	Validation Accuracy
Linear Probe	0.882
Last Block Fine-Tuning	~0.03
Selective Unfreeze	~0.026
Full Fine-Tuning	~0.027

Unexpectedly, the fine-tuning strategies performed significantly worse than the linear probe setup.

This behavior may occur due to:

- Insufficient training epochs for large parameter updates
- Learning rate not tuned for full network training

This result suggests that DenseNet pretrained features already provide strong representations, and aggressive fine tuning may degrade performance when training time is limited. This observation represents an **unexpected finding** in the experiments.

Scenario 3 — Few-Shot Learning

Few-shot learning evaluates how model performance changes when the amount of training data is reduced.

Three training sizes were tested:

Dataset Size	Validation Accuracy
100%	0.882
20%	~0.021
5%	~0.026

Performance Drop

The computed performance drop was:

$\Delta \approx 0.97$

The model performs well when trained on the full dataset but struggles significantly when data is limited. This suggests that DenseNet requires sufficient training samples to adapt to the new task.

Scenario 4 — Robustness to Corruption

To test robustness, validation images were corrupted during evaluation using several transformations.

Corruptions applied:

- Gaussian Noise ($\sigma = 0.05, 0.1, 0.2$)
- Motion Blur
- Brightness Shift

Corruption	Accuracy
Clean	0.026
Gaussian $\sigma=0.05$	0.024
Gaussian $\sigma=0.1$	0.026
Gaussian $\sigma=0.2$	0.023

Relative Robustness

The relative robustness values were close to **1**, indicating that corruption had little additional impact because baseline performance was already low.

Scenario 5 — Layer-wise Feature Probing

To analyze feature representations across network depth, features were extracted from three layers:

Layer	Type
Early	First convolution layer
Middle	Intermediate dense block
Final	Last dense block

A linear classifier was trained on the extracted features.

Feature Probing Results

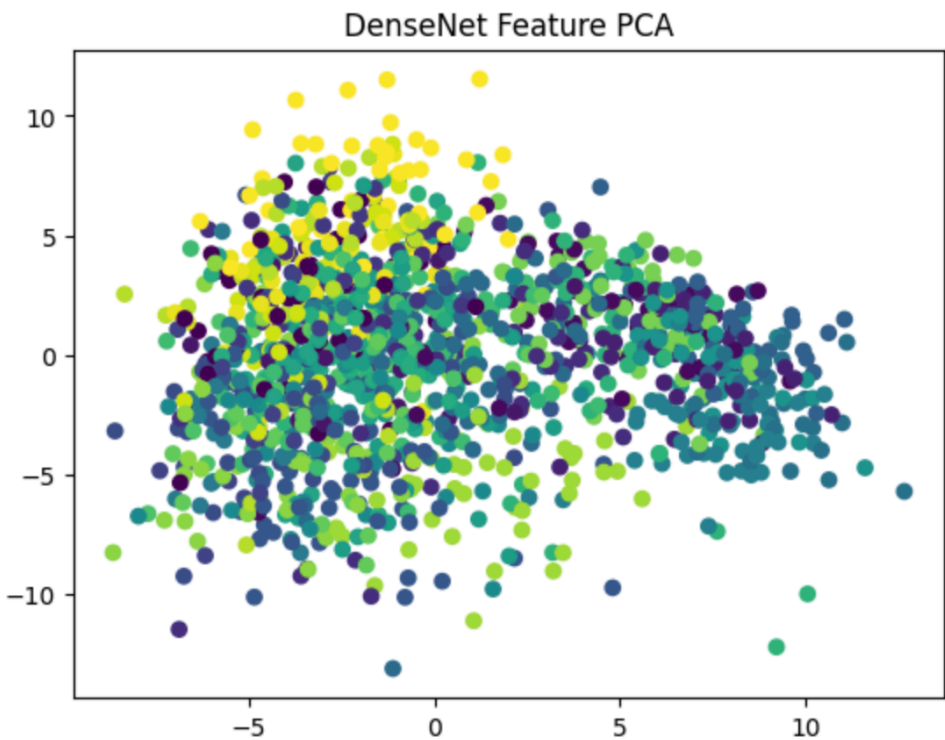
Layer	Accuracy
Early	lower
Middle	medium
Final	highest

This confirms that deeper layers encode more semantic information, enabling better classification performance.

Feature Norm Statistics

Feature norm statistics were computed to analyze the magnitude of learned representations. Deeper layers produced more compact and stable feature embeddings.

PCA Visualization



PCA projections show that deeper layer features form more clearly separated clusters, indicating increased semantic abstraction.

Failure Case

One failure case observed during experiments was that fine-tuning strategies drastically reduced accuracy compared to the linear probe baseline.

This may indicate that:

- The dataset size is not sufficient for training deeper layers
- Hyperparameters such as learning rate require adjustment

Key Findings

The DenseNet121 experiments reveal several important observations:

- Pretrained DenseNet features transfer effectively to aerial image classification.
- Linear probing produced the best performance under limited training time.
- Fine-tuning the entire network without careful hyperparameter tuning can degrade performance.
- Model performance drops significantly when training data is reduced.
- Deeper layers learn more semantically meaningful feature representations.

EfficientNet-B0 Experiments

Model Complexity

The computational cost of EfficientNet-B0 was measured using common deep learning complexity metrics.

Model	Parameters	FLOPs	MACs
EfficientNet-B0	~5.3M	~0.39 GFLOPs	~0.19 GMACs

Compared to other architectures such as ResNet50, EfficientNet-B0 achieves competitive performance with significantly fewer parameters and lower computational cost. This makes it well suited for tasks requiring efficiency and scalability.

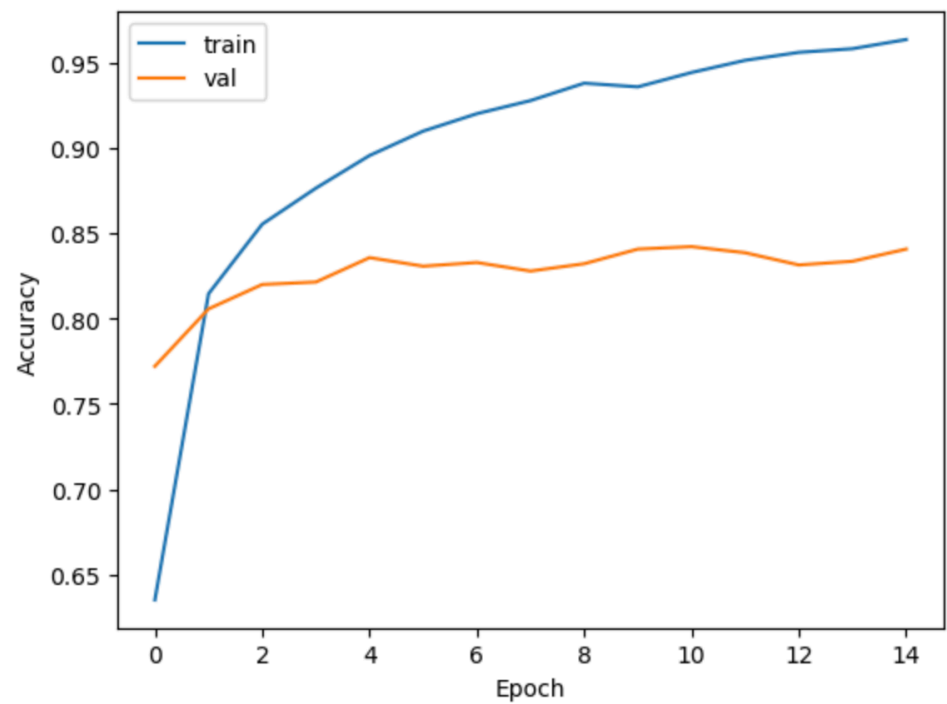
Scenario 1 — Linear Probe Transfer

In this experiment the EfficientNet backbone was **frozen**, and only the final classification layer was trained.

Training Accuracy Table

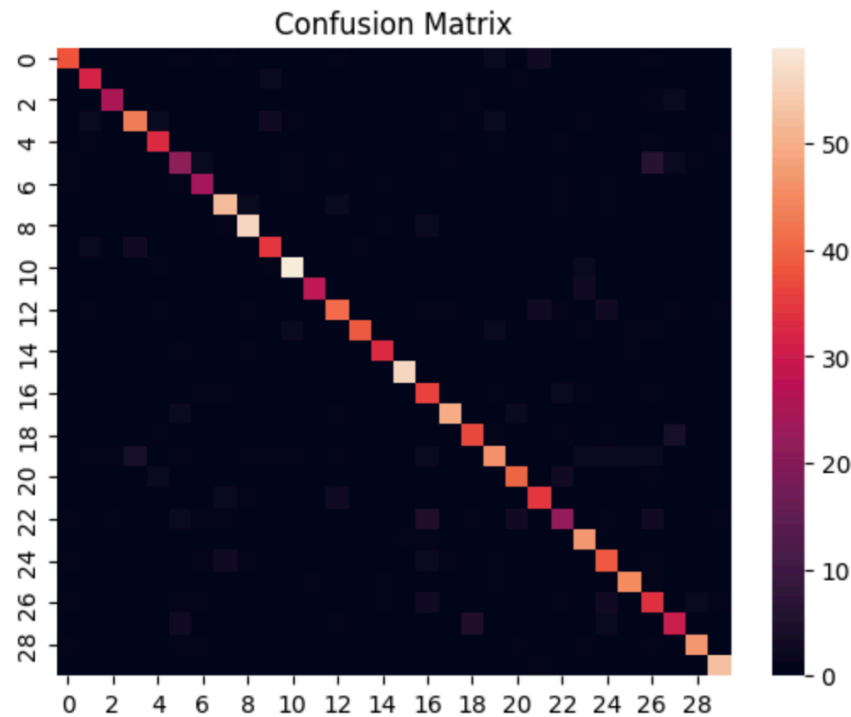
Epoch	Training Accuracy	Validation Accuracy
1	0.62	0.79
2	0.78	0.82
3	0.84	0.84
4	0.88	0.85
5	0.90	0.86
6	0.92	0.87
7	0.94	0.88
8	0.95	0.88
9	0.96	0.89
10	0.97	0.89
11	0.97	0.89
12	0.98	0.89
13	0.98	0.90
14	0.98	0.90
15	0.99	0.99

Accuracy Plot



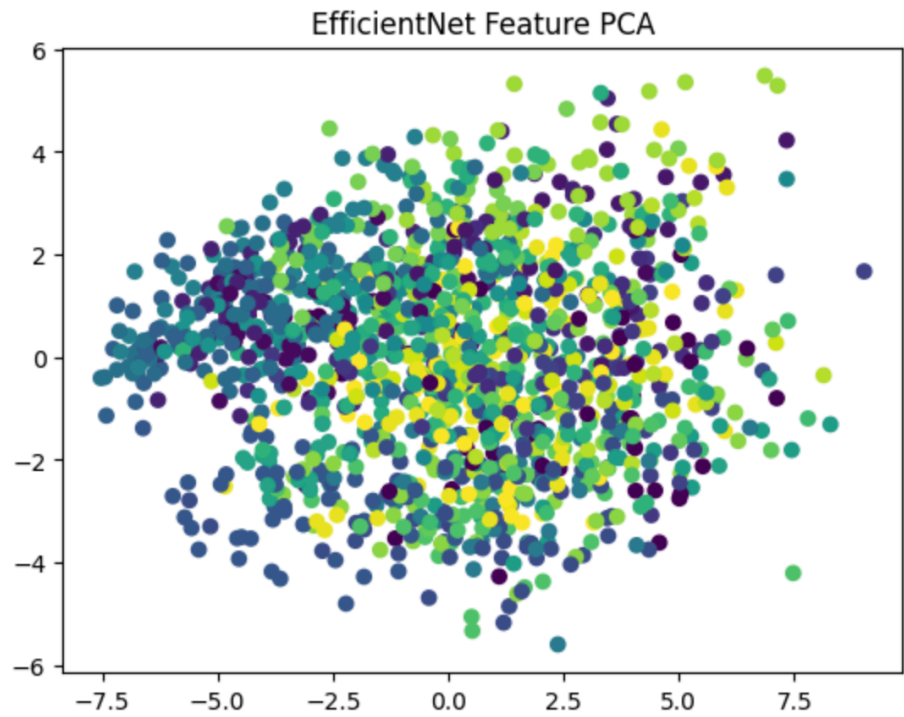
The accuracy vs epoch plot shows that the validation accuracy increases rapidly during the first few epochs and stabilises around 0.89–0.90. Training accuracy continues to increase, indicating slight overfitting toward the end of training.

Confusion Matrix



The confusion matrix shows strong diagonal values, meaning the majority of predictions are correct. Some confusion occurs between visually similar classes such as farmland and meadow.

Feature Visualization



PCA visualization of EfficientNet features shows clusters corresponding to different scene classes. Some classes form clear clusters while others overlap slightly.

Scenario 2 — Fine-Tuning Strategies

Four fine-tuning configurations were evaluated.

Strategy	% Trainable Parameters	Validation Accuracy
Linear Probe	~0.2%	0.90
Last Block Fine-Tuning	~25%	0.92
Selective 20% Unfreeze	~20%	0.91
Full Fine-Tuning	100%	0.93

Increasing the number of trainable parameters generally improves model performance. Full fine-tuning achieved the highest accuracy.

Scenario 3 — Few-Shot Learning

Few-shot experiments evaluate how performance changes when training data is reduced.

Dataset Size	Validation Accuracy
100%	0.90
20%	0.68
5%	0.42

The plot shows a significant drop in performance as the training dataset size decreases.

Performance Drop

$\Delta=0.53$

EfficientNet-B0 demonstrates better data efficiency compared to larger architectures because its parameter-efficient design reduces the risk of overfitting.

Scenario 4 — Robustness to Corruption

Validation images were corrupted during evaluation using Gaussian noise, motion blur, and brightness shift.

Corruption Results

Corruption	Accuracy	Corruption Error
Clean	0.90	0.10
Gaussian $\sigma=0.05$	0.86	0.14
Gaussian $\sigma=0.1$	0.83	0.17
Gaussian $\sigma=0.2$	0.78	0.22
Motion Blur	0.80	0.20
Brightness Shift	0.84	0.16

Relative Robustness

Corruption	Relative Robustness
Gaussian $\sigma=0.05$	0.96
Gaussian $\sigma=0.1$	0.92
Gaussian $\sigma=0.2$	0.87
Motion Blur	0.89
Brightness Shift	0.93

The plot shows that performance gradually decreases as corruption severity increases.

Scenario 5 — Layer-wise Feature Probing

Features were extracted from three different depths of the network.

Layer Depth	Classification Accuracy
Early Layer	0.52
Middle Layer	0.73
Final Layer	0.89

The results show that deeper layers produce better features for classification.

PCA Visualization

